

# QuickBite — Projet fil rouge : étapes de création & ateliers (avec code)

Formateur : **Haithem Mihoubi** — Formation Agilité / DevOps / Spring Security. Ce document décrit, **atelier par atelier**, comment construire ce projet **de zéro jusqu'au système sécurisé complet**. Le **code complet de chaque fichier** est inclus dans la section de l'atelier qui le crée. À chaque atelier : on applique une notion du cours, on ajoute le code, puis on **teste**.

## Table des ateliers

| #  | Atelier                                      | Notion appliquée                               | Module    |
|----|----------------------------------------------|------------------------------------------------|-----------|
| 0  | Squelette Spring Boot + <code>/health</code> | Projet, structure                              | 3 — J1    |
| 1  | Sécurité de base                             | AuthN/AuthZ, SecurityFilterChain               | 3 — J1    |
| 2  | Utilisateurs en base + RBAC                  | JPA, UserDetailsService, BCrypt, @PreAuthorize | 3 — J2    |
| 3  | Authentification JWT (login/refresh)         | JWT, stateless, filtre, 401/403                | 3 — J3    |
| 4  | OAuth2 / Keycloak (optionnel)                | OAuth2, OIDC, resource server                  | 3 — J4    |
| 5  | Durcissement                                 | CORS, exceptions, rate limit, headers          | 3 — J5    |
| D1 | Conteneurisation Docker                      | Image, Dockerfile, Compose                     | 2 — J2    |
| D2 | Pipeline CI                                  | Intégration continue                           | 2 — J1/J4 |
| D3 | Déploiement Kubernetes                       | Pods, Deployment, Service, Ingress             | 2 — J3/J4 |

## Pré-requis

- **JDK 21**, **Maven 3.8+**, un IDE, **Postman** (collection fournie), et pour la partie DevOps : **Docker**, **kubectl**, **Minikube**.

## Démarrage & comptes de test

```
mvn spring-boot:run
curl http://localhost:8080/health
```

| Utilisateur | Mot de passe | Rôle   |
|-------------|--------------|--------|
| admin       | admin123     | ADMIN  |
| client      | client123    | CLIENT |

---

# Atelier 0 — Squelette Spring Boot

---

**Objectif :** un projet Spring Boot 3 qui démarre et expose `/health` (public).

Fichier : pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.3.4</version>
    <relativePath/>
  </parent>

  <groupId>com.quickbite</groupId>
  <artifactId>quickbite-api</artifactId>
  <version>1.0.0</version>
  <name>QuickBite API</name>

  <properties>
    <java.version>21</java.version>
    <jjwt.version>0.12.6</jjwt.version>
  </properties>

  <dependencies>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-security</artifactId>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-data-jpa</artifactId>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-validation</artifactId>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-actuator</artifactId>
    </dependency>

    <dependency>
      <groupId>com.h2database</groupId>
      <artifactId>h2</artifactId>
      <scope>runtime</scope>
    </dependency>
    <dependency>
      <groupId>org.postgresql</groupId>
      <artifactId>postgresql</artifactId>
      <scope>runtime</scope>
    </dependency>

    <dependency>
      <groupId>io.jsonwebtoken</groupId>
      <artifactId>jjwt-api</artifactId>
      <version>${jjwt.version}</version>
    </dependency>
  </dependencies>
</project>
```

```
</dependency>
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-impl</artifactId>
  <version>${jjwt.version}</version>
  <scope>runtime</scope>
</dependency>
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-jackson</artifactId>
  <version>${jjwt.version}</version>
  <scope>runtime</scope>
</dependency>

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-test</artifactId>
  <scope>test</scope>
</dependency>
<dependency>
  <groupId>org.springframework.security</groupId>
  <artifactId>spring-security-test</artifactId>
  <scope>test</scope>
</dependency>
</dependencies>

<build>
  <plugins>
    <plugin>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-maven-plugin</artifactId>
    </plugin>
  </plugins>
</build>
</project>
```

**Fichier :** `src/main/resources/application.yml`

```
spring:
  application:
    name: quickbite-api
  datasource:
    url: jdbc:h2:mem:quickbite;DB_CLOSE_DELAY=-1
    username: sa
    password: ""
    driver-class-name: org.h2.Driver
  jpa:
    hibernate:
      ddl-auto: create-drop
      open-in-view: false
  h2:
    console:
      enabled: true          # http://localhost:8080/h2-console (dev uniquement)

server:
  port: 8080

quickbite:
  security:
    jwt:
      secret: ${JWT_SECRET:change-me-please-32-bytes-minimum-secret-key!!}
      access-token-minutes: 15
      refresh-token-days: 7
    cors:
      allowed-origins: ${CORS_ORIGINS:http://localhost:4200,http://localhost:3000}

management:
  endpoints:
    web:
      exposure:
        include: health,info
  endpoint:
    health:
      show-details: when_authorized
```

**Fichier :** `src/main/java/com/quickbite/QuickBiteApplication.java`

```
package com.quickbite;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class QuickBiteApplication {
    public static void main(String[] args) {
        SpringApplication.run(QuickBiteApplication.class, args);
    }
}
```

**Fichier :** `src/main/java/com/quickbite/web/HealthController.java`

```
package com.quickbite.web;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

import java.time.Instant;
import java.util.Map;

@RestController
public class HealthController {

    @GetMapping("/health")
    public Map<String, Object> health() {
        return Map.of(
            "status", "UP",
            "service", "quickbite-api",
            "time", Instant.now().toString()
        );
    }
}
```

**Test :**

```
curl http://localhost:8080/health
# {"status":"UP","service":"quickbite-api", ...}
```



# Atelier 1 — Sécurité de base

**Notion (M3-J1)** : Spring Security protège **tout** par défaut. On découvre la `SecurityFilterChain` (Spring Security 6) et on déclare ce qui est public.

Version **initiale** de la config (elle sera enrichie aux ateliers 2, 3 et 5). On la remplace ensuite par la version complète donnée à l'atelier 3.

```
package com.quickbite.security;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http.authorizeHttpRequests(auth -> auth
            .requestMatchers("/health").permitAll()
            .anyRequest().authenticated());
        return http.build();
    }
}
```

**Test :**

```
curl -i http://localhost:8080/auth/me # 401 sans authentication
```

## Atelier 2 — Utilisateurs en base + RBAC

**Notion (M3-J2)** : authentication via la base, `UserDetailsService` personnalisé, hachage **BCrypt**, autorisation par rôles avec `@PreAuthorize`.

**Piège classique** : `hasRole('ADMIN')` exige l'autorité `ROLE_ADMIN` (préfixe `ROLE_` implicite). Rappel codes : **401** = non authentifié, **403** = authentifié mais sans droit.

**Fichier** : `user/RoleEntity.java`

```
package com.quickbite.user;

import jakarta.persistence.*;

@Entity
@Table(name = "roles")
public class RoleEntity {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(unique = true, nullable = false)
    private String name;

    protected RoleEntity() { }
    public RoleEntity(String name) { this.name = name; }

    public Long getId() { return id; }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
}
```

**Fichier :** `user/UserEntity.java`

```
package com.quickbite.user;

import jakarta.persistence.*;
import java.util.HashSet;
import java.util.Set;

@Entity
@Table(name = "users")
public class UserEntity {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(unique = true, nullable = false)
    private String username;

    @Column(nullable = false)
    private String password;           // HASH BCrypt, jamais en clair

    @ManyToMany(fetch = FetchType.EAGER)
    @JoinTable(name = "user_roles",
        joinColumns = @JoinColumn(name = "user_id"),
        inverseJoinColumns = @JoinColumn(name = "role_id"))
    private Set<RoleEntity> roles = new HashSet<>();

    protected UserEntity() { }
    public UserEntity(String username, String password) {
        this.username = username;
        this.password = password;
    }

    public void addRole(RoleEntity role) { this.roles.add(role); }

    public Long getId() { return id; }
    public String getUsername() { return username; }
    public void setUsername(String username) { this.username = username; }
    public String getPassword() { return password; }
    public void setPassword(String password) { this.password = password; }
    public Set<RoleEntity> getRoles() { return roles; }
    public void setRoles(Set<RoleEntity> roles) { this.roles = roles; }
}
```

**Fichier :** `user/UserRepository.java`

```
package com.quickbite.user;

import org.springframework.data.jpa.repository.JpaRepository;
import java.util.Optional;

public interface UserRepository extends JpaRepository<UserEntity, Long> {
    Optional<UserEntity> findByUsername(String username);
    boolean existsByUsername(String username);
}
```

**Fichier :** `user/RoleRepository.java`

```
package com.quickbite.user;

import org.springframework.data.jpa.repository.JpaRepository;
import java.util.Optional;

public interface RoleRepository extends JpaRepository<RoleEntity, Long> {
    Optional<RoleEntity> findByName(String name);
}
```

**Fichier :** `user/JpaUserDetailsService.java`

```
package com.quickbite.user;

import org.springframework.security.core.authority.SimpleGrantedAuthority;
import org.springframework.security.core.userdetails.User;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.core.userdetails.UsernameNotFoundException;
import org.springframework.stereotype.Service;
import java.util.List;

@Service
public class JpaUserDetailsService implements UserDetailsService {

    private final UserRepository repo;
    public JpaUserDetailsService(UserRepository repo) { this.repo = repo; }

    @Override
    public UserDetails loadUserByUsername(String username) {
        UserEntity u = repo.findByUsername(username)
            .orElseThrow(() -> new UsernameNotFoundException("Identifiants invalides"));

        List<SimpleGrantedAuthority> authorities = u.getRoles().stream()
            .map(r -> new SimpleGrantedAuthority("ROLE_" + r.getName()))
            .toList();

        return User.withUsername(u.getUsername())
            .password(u.getPassword())
            .authorities(authorities)
            .build();
    }
}
```

**Fichier : `config/DataInitializer.java` (jeu de données admin/client)**

```
package com.quickbite.config;

import com.quickbite.order.OrderEntity;
import com.quickbite.order.OrderRepository;
import com.quickbite.user.*;
import org.springframework.boot.CommandLineRunner;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.crypto.password.PasswordEncoder;

@Configuration
public class DataInitializer {

    @Bean
    CommandLineRunner seed(UserRepository users, RoleRepository roles,
                           OrderRepository orders, PasswordEncoder encoder) {
        return args -> {
            if (users.count() > 0) return;

            RoleEntity admin = roles.save(new RoleEntity("ADMIN"));
            RoleEntity client = roles.save(new RoleEntity("CLIENT"));

            UserEntity a = new UserEntity("admin", encoder.encode("admin123"));
            a.addRole(admin);
            users.save(a);

            UserEntity c = new UserEntity("client", encoder.encode("client123"));
            c.addRole(client);
            users.save(c);

            orders.save(new OrderEntity("Pizza Margherita", 9.50, "client"));
            orders.save(new OrderEntity("Burger Veggie", 8.00, "client"));
        };
    }
}
```

**Fichier :** `order/OrderEntity.java`

```
package com.quickbite.order;

import jakarta.persistence.*;

@Entity
@Table(name = "orders")
public class OrderEntity {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false) private String item;
    @Column(nullable = false) private double price;
    @Column(nullable = false) private String owner; // username du propriétaire

    protected OrderEntity() { }
    public OrderEntity(String item, double price, String owner) {
        this.item = item; this.price = price; this.owner = owner;
    }

    public Long getId() { return id; }
    public String getItem() { return item; }
    public void setItem(String item) { this.item = item; }
    public double getPrice() { return price; }
    public void setPrice(double price) { this.price = price; }
    public String getOwner() { return owner; }
    public void setOwner(String owner) { this.owner = owner; }
}
```

**Fichier :** `order/OrderRepository.java`

```
package com.quickbite.order;

import org.springframework.data.jpa.repository.JpaRepository;
import java.util.List;

public interface OrderRepository extends JpaRepository<OrderEntity, Long> {
    List<OrderEntity> findByOwner(String owner);
}
```

**Fichier : `order/OrderController.java` (RBAC avec `@PreAuthorize`)**

```
package com.quickbite.order;

import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Positive;
import org.springframework.http.HttpStatus;
import org.springframework.security.access.prepost.PreAuthorize;
import org.springframework.security.core.Authentication;
import org.springframework.web.bind.annotation.*;
import org.springframework.web.server.ResponseStatusException;
import java.util.List;

@RestController
@RequestMapping("/orders")
public class OrderController {

    private final OrderRepository repo;
    public OrderController(OrderRepository repo) { this.repo = repo; }

    public record CreateOrder(@NotBlank String item, @Positive double price) { }

    @GetMapping
    @PreAuthorize("isAuthenticated()")
    public List<OrderEntity> myOrders(Authentication auth) {
        return repo.findByOwner(auth.getName());
    }

    @PostMapping
    @PreAuthorize("hasRole('CLIENT') or hasRole('ADMIN')")
    @ResponseStatus(HttpStatus.CREATED)
    public OrderEntity create(@RequestBody CreateOrder body, Authentication auth) {
        return repo.save(new OrderEntity(body.item(), body.price(), auth.getName()));
    }

    @DeleteMapping("/{id}")
    @PreAuthorize("hasRole('ADMIN')")
    @ResponseStatus(HttpStatus.NO_CONTENT)
    public void delete(@PathVariable Long id) {
        if (!repo.existsById(id)) {
            throw new ResponseStatusException(HttpStatus.NOT_FOUND, "Commande introuvable");
        }
        repo.deleteById(id);
    }
}
```

**Fichier :** `admin/AdminController.java`

```
package com.quickbite.admin;

import com.quickbite.user.UserRepository;
import org.springframework.security.access.prepost.PreAuthorize;
import org.springframework.web.bind.annotation.*;
import java.util.List;
import java.util.Map;

@RestController
@RequestMapping("/admin")
@PreAuthorize("hasRole('ADMIN')") // défense en profondeur (en plus de l'URL /admin/**)
public class AdminController {

    private final UserRepository userRepo;
    public AdminController(UserRepository userRepo) { this.userRepo = userRepo; }

    @GetMapping("/users")
    public List<Map<String, Object>> users() {
        return userRepo.findAll().stream()
            .map(u -> Map.<String, Object>.of(
                "id", u.getId(),
                "username", u.getUsername(),
                "roles", u.getRoles().stream().map(r -> r.getName()).toList()))
            .toList();
    }
}
```



## Atelier 3 — Authentification JWT

**Notion (M3-J3) :** API **stateless** sécurisée par **JWT**. Access token court + refresh token long avec **rotation**. Le JWT est **signé** (intégrité) mais **pas chiffré** : aucune donnée sensible dedans.

**Fichier :** `security/SecurityProperties.java`

```
package com.quickbite.security;

import org.springframework.boot.context.properties.ConfigurationProperties;
import java.util.List;

@ConfigurationProperties(prefix = "quickbite.security")
public class SecurityProperties {

    private final Jwt jwt = new Jwt();
    private final Cors cors = new Cors();
    public Jwt getJwt() { return jwt; }
    public Cors getCors() { return cors; }

    public static class Jwt {
        private String secret;
        private long accessTokenMinutes = 15;
        private long refreshTokenDays = 7;
        public String getSecret() { return secret; }
        public void setSecret(String s) { this.secret = s; }
        public long getAccessTokenMinutes() { return accessTokenMinutes; }
        public void setAccessTokenMinutes(long v) { this.accessTokenMinutes = v; }
        public long getRefreshTokenDays() { return refreshTokenDays; }
        public void setRefreshTokenDays(long v) { this.refreshTokenDays = v; }
    }

    public static class Cors {
        private List<String> allowedOrigins = List.of();
        public List<String> getAllowedOrigins() { return allowedOrigins; }
        public void setAllowedOrigins(List<String> v) { this.allowedOrigins = v; }
    }
}
```

**Fichier :** security/JwtService.java

```
package com.quickbite.security;

import io.jsonwebtoken.Claims;
import io.jsonwebtoken.Jws;
import io.jsonwebtoken.Jwts;
import io.jsonwebtoken.security.Keys;
import org.springframework.security.core.GrantedAuthority;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.stereotype.Service;
import javax.crypto.SecretKey;
import java.nio.charset.StandardCharsets;
import java.util.Date;
import java.util.List;

@Service
public class JwtService {

    private final SecretKey key;
    private final long accessMillis;

    public JwtService(SecurityProperties props) {
        this.key = Keys.hmacShaKeyFor(props.getJwt().getSecret().getBytes(StandardCharsets.UTF_8));
        this.accessMillis = props.getJwt().getAccessTokenMinutes() * 60_000;
    }

    public String generateAccess(UserDetails user) {
        Date now = new Date();
        return Jwts.builder()
            .subject(user.getUsername())
            .claim("roles", user.getAuthorities().stream()
                .map(GrantedAuthority::getAuthority).toList())
            .issuedAt(now)
            .expiration(new Date(now.getTime() + accessMillis))
            .signWith(key)
            .compact();
    }

    public Jws<Claims> parse(String token) {
        return Jwts.parser().verifyWith(key).build().parseSignedClaims(token);
    }

    @SuppressWarnings("unchecked")
    public List<String> extractRoles(Claims claims) {
        Object roles = claims.get("roles");
        return roles instanceof List<?> list ? (List<String>) list : List.of();
    }
}
```

**Fichier :** `security/JwtAuthFilter.java`

```
package com.quickbite.security;

import io.jsonwebtoken.Claims;
import io.jsonwebtoken.JwtException;
import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import org.springframework.lang.NonNull;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.authority.SimpleGrantedAuthority;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.stereotype.Component;
import org.springframework.web.filter.OncePerRequestFilter;
import java.io.IOException;

@Component
public class JwtAuthFilter extends OncePerRequestFilter {

    private final JwtService jwt;
    public JwtAuthFilter(JwtService jwt) { this.jwt = jwt; }

    @Override
    protected void doFilterInternal(@NonNull HttpServletRequest req,
                                    @NonNull HttpServletResponse res,
                                    @NonNull FilterChain chain)
        throws ServletException, IOException {

        String header = req.getHeader("Authorization");
        if (header != null && header.startsWith("Bearer ")) {
            try {
                Claims claims = jwt.parse(header.substring(7)).getPayload();
                var authorities = jwt.extractRoles(claims).stream()
                    .map(SimpleGrantedAuthority::new).toList();
                var auth = new UsernamePasswordAuthenticationToken(
                    claims.getSubject(), null, authorities);
                SecurityContextHolder.getContext().setAuthentication(auth);
            } catch (JwtException | IllegalArgumentException e) {
                SecurityContextHolder.clearContext(); // token invalide/expiré
            }
        }
        chain.doFilter(req, res);
    }
}
```

## Fichiers : `security/JwtEntryPoint.java` (401) & `security/RestAccessDeniedHandler.java` (403)

```
package com.quickbite.security;

import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import org.springframework.security.core.AuthenticationException;
import org.springframework.security.web.AuthenticationEntryPoint;
import org.springframework.stereotype.Component;
import java.io.IOException;

@Component
public class JwtEntryPoint implements AuthenticationEntryPoint {
    @Override
    public void commence(HttpServletRequest req, HttpServletResponse res,
                        AuthenticationException ex) throws IOException {
        res.setContentType("application/json");
        res.setStatus(HttpServletResponse.SC_UNAUTHORIZED);
        res.getWriter().write("{\"error\": \"Authentification requise\"}");
    }
}
```

```
package com.quickbite.security;

import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import org.springframework.security.access.AccessDeniedException;
import org.springframework.security.web.access.AccessDeniedHandler;
import org.springframework.stereotype.Component;
import java.io.IOException;

@Component
public class RestAccessDeniedHandler implements AccessDeniedHandler {
    @Override
    public void handle(HttpServletRequest req, HttpServletResponse res,
                    AccessDeniedException ex) throws IOException {
        res.setContentType("application/json");
        res.setStatus(HttpServletResponse.SC_FORBIDDEN);
        res.getWriter().write("{\"error\": \"Accès refusé\"}");
    }
}
```

**Fichiers :** `auth/RefreshTokenEntity.java` & `auth/RefreshTokenRepository.java`

```
package com.quickbite.auth;

import jakarta.persistence.*;
import java.time.Instant;

@Entity
@Table(name = "refresh_tokens")
public class RefreshTokenEntity {

    @Id @Column(length = 64)
    private String token; // UUID
    @Column(nullable = false) private String username;
    @Column(nullable = false) private Instant expiresAt;
    @Column(nullable = false) private boolean revoked = false;

    protected RefreshTokenEntity() { }
    public RefreshTokenEntity(String token, String username, Instant expiresAt) {
        this.token = token; this.username = username; this.expiresAt = expiresAt;
    }

    public boolean isValid() { return !revoked && expiresAt.isAfter(Instant.now()); }

    public String getToken() { return token; }
    public String getUsername() { return username; }
    public Instant getExpiresAt() { return expiresAt; }
    public boolean isRevoked() { return revoked; }
    public void setRevoked(boolean revoked) { this.revoked = revoked; }
}
```

```
package com.quickbite.auth;

import org.springframework.data.jpa.repository.JpaRepository;
import java.util.Optional;

public interface RefreshTokenRepository extends JpaRepository<RefreshTokenEntity, String> {
    Optional<RefreshTokenEntity> findByToken(String token);
}
```

**Fichier :** `auth/AuthDtos.java`

```
package com.quickbite.auth;

import jakarta.validation.constraints.NotBlank;

public class AuthDtos {

    public record LoginRequest(@NotBlank String username, @NotBlank String password) { }
    public record RefreshRequest(@NotBlank String refreshToken) { }
    public record TokenResponse(String accessToken, String refreshToken, String tokenType) {
        public static TokenResponse of(String access, String refresh) {
            return new TokenResponse(access, refresh, "Bearer");
        }
    }
}
```

**Fichier :** `auth/AuthService.java` (login + refresh avec rotation)

```
package com.quickbite.auth;

import com.quickbite.auth.AuthDtos.TokenResponse;
import com.quickbite.security.JwtService;
import com.quickbite.security.SecurityProperties;
import org.springframework.security.authentication.AuthenticationManager;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import java.time.Instant;
import java.time.temporal.ChronoUnit;
import java.util.UUID;

@Service
public class AuthService {

    private final AuthenticationManager authManager;
    private final UserDetailsService userDetailsService;
    private final JwtService jwtService;
    private final RefreshTokenRepository refreshRepo;
    private final long refreshDays;

    public AuthService(AuthenticationManager authManager, UserDetailsService uds,
                      JwtService jwtService, RefreshTokenRepository refreshRepo,
                      SecurityProperties props) {
        this.authManager = authManager;
        this.userDetailsService = uds;
        this.jwtService = jwtService;
        this.refreshRepo = refreshRepo;
        this.refreshDays = props.getJwt().getRefreshTokenDays();
    }

    @Transactional
    public TokenResponse login(String username, String password) {
        authManager.authenticate(new UsernamePasswordAuthenticationToken(username, password));
        UserDetails user = userDetailsService.loadUserByUsername(username);
        return TokenResponse.of(jwtService.generateAccess(user), createRefreshToken(username));
    }

    @Transactional
    public TokenResponse refresh(String refreshToken) {
        RefreshTokenEntity stored = refreshRepo.findByToken(refreshToken)
            .filter(RefreshTokenEntity::isValid)
            .orElseThrow(() -> new IllegalArgumentException("Refresh token invalide ou expiré"));

        stored.setRevoked(true); // rotation : on révoque l'ancien
        refreshRepo.save(stored);

        UserDetails user = userDetailsService.loadUserByUsername(stored.getUsername());
        return TokenResponse.of(jwtService.generateAccess(user),
                               createRefreshToken(stored.getUsername()));
    }

    private String createRefreshToken(String username) {
        String token = UUID.randomUUID().toString();
        Instant expiry = Instant.now().plus(refreshDays, ChronoUnit.DAYS);
        refreshRepo.save(new RefreshTokenEntity(token, username, expiry));
        return token;
    }
}
```



```
}  
}
```

**Fichier :** `auth/AuthController.java`

```
package com.quickbite.auth;  
  
import com.quickbite.auth.AuthDtos.*;  
import jakarta.validation.Valid;  
import org.springframework.security.core.Authentication;  
import org.springframework.web.bind.annotation.*;  
import java.util.Map;  
  
@RestController  
@RequestMapping("/auth")  
public class AuthController {  
  
    private final AuthService authService;  
    public AuthController(AuthService authService) { this.authService = authService; }  
  
    @PostMapping("/login")  
    public TokenResponse login(@Valid @RequestBody LoginRequest req) {  
        return authService.login(req.username(), req.password());  
    }  
  
    @PostMapping("/refresh")  
    public TokenResponse refresh(@Valid @RequestBody RefreshRequest req) {  
        return authService.refresh(req.refreshToken());  
    }  
  
    @GetMapping("/me")  
    public Map<String, Object> me(Authentication auth) {  
        return Map.of("username", auth.getName(), "authorities", auth.getAuthorities());  
    }  
}
```

**Fichier :** `security/SecurityConfig.java` — **version complète (remplace celle de l'atelier 1)**

```

package com.quickbite.security;

import org.springframework.boot.context.properties.EnableConfigurationProperties;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.authentication.AuthenticationManager;
import org.springframework.security.config.annotation.authentication.configuration.AuthenticationConfiguration;
import org.springframework.security.config.annotation.method.configuration.EnableMethodSecurity;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter;
import org.springframework.web.cors.*;
import java.util.List;

@Configuration
@EnableWebSecurity
@EnableMethodSecurity // active @PreAuthorize (atelier 2)
@EnableConfigurationProperties(SecurityProperties.class)
public class SecurityConfig {

    private final JwtAuthFilter jwtAuthFilter;
    private final LoginRateLimitFilter loginRateLimitFilter;
    private final JwtEntryPoint entryPoint;
    private final RestAccessDeniedHandler accessDeniedHandler;
    private final SecurityProperties props;

    public SecurityConfig(JwtAuthFilter jwtAuthFilter, LoginRateLimitFilter loginRateLimitFilter,
                        JwtEntryPoint entryPoint, RestAccessDeniedHandler accessDeniedHandler,
                        SecurityProperties props) {
        this.jwtAuthFilter = jwtAuthFilter;
        this.loginRateLimitFilter = loginRateLimitFilter;
        this.entryPoint = entryPoint;
        this.accessDeniedHandler = accessDeniedHandler;
        this.props = props;
    }

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http
            .cors(cors -> cors.configurationSource(corsSource())) // atelier 5
            .csrf(csrf -> csrf.disable()) // API par token
            .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/health", "/actuator/health").permitAll()
                .requestMatchers("/auth/login", "/auth/refresh").permitAll()
                .requestMatchers("/h2-console/**").permitAll() // dev uniquement
                .requestMatchers("/admin/**").hasRole("ADMIN")
                .anyRequest().authenticated()
            )
            .exceptionHandling(e -> e
                .authenticationEntryPoint(entryPoint) // 401
                .accessDeniedHandler(accessDeniedHandler) // 403
            )
            .headers(h -> h.frameOptions(f -> f.sameOrigin())) // atelier 5
            .addFilterBefore(loginRateLimitFilter, UsernamePasswordAuthenticationFilter.class)
            .addFilterBefore(jwtAuthFilter, UsernamePasswordAuthenticationFilter.class);
        return http.build();
    }
}

```

```

@Bean
public PasswordEncoder passwordEncoder() { return new BCryptPasswordEncoder(); }

@Bean
public AuthenticationManager authenticationManager(AuthenticationConfiguration cfg) throws Exception {
    return cfg.getAuthenticationManager();
}

@Bean
public CorsConfigurationSource corsSource() {
    CorsConfiguration c = new CorsConfiguration();
    c.setAllowedOrigins(props.getCors().getAllowedOrigins());
    c.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE", "OPTIONS"));
    c.setAllowedHeaders(List.of("Authorization", "Content-Type"));
    c.setAllowCredentials(true);
    UrlBasedCorsConfigurationSource src = new UrlBasedCorsConfigurationSource();
    src.registerCorsConfiguration("/**", c);
    return src;
}
}

```

## Endpoints & bodies :

| Méthode | URL           | Accès        | Body                                          |
|---------|---------------|--------------|-----------------------------------------------|
| POST    | /auth/login   | public       | {"username": "admin", "password": "admin123"} |
| POST    | /auth/refresh | public       | {"refreshToken": "<uuid>"}                    |
| GET     | /auth/me      | authentifié  | —                                             |
| GET     | /orders       | authentifié  | —                                             |
| POST    | /orders       | CLIENT/ADMIN | {"item": "Tacos", "price": 7.5}               |
| DELETE  | /orders/{id}  | ADMIN        | —                                             |
| GET     | /admin/users  | ADMIN        | —                                             |

## Test :

```

B=http://localhost:8080
AT=$(curl -s -X POST $B/auth/login -H "Content-Type: application/json" \
  -d '{"username": "admin", "password": "admin123"}' | jq -r .accessToken)
curl -s $B/auth/me -H "Authorization: Bearer $AT"
curl -s $B/admin/users -H "Authorization: Bearer $AT" # 200
CT=$(curl -s -X POST $B/auth/login -H "Content-Type: application/json" \
  -d '{"username": "client", "password": "client123"}' | jq -r .accessToken)
curl -s -o /dev/null -w "%{http_code}\n" $B/admin/users -H "Authorization: Bearer $CT" # 403

```

## Atelier 4 — OAuth2 / Keycloak (optionnel)

**Notion (M3-J4) :** déléguer l'authentification à **Keycloak** ; l'API devient un **resource server**.

```
# 1) Lancer Keycloak
docker run -p 8081:8080 -e KEYCLOAK_ADMIN=admin -e KEYCLOAK_ADMIN_PASSWORD=admin \
  quay.io/keycloak/keycloak:24.0 start-dev
# 2) Créer realm "quickbite", un client, les rôles admin/client, des utilisateurs.
```

Dépendance à ajouter dans `pom.xml` :

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-oauth2-resource-server</artifactId>
</dependency>
```

Configuration (profil dédié) :

```
spring:
  security:
    oauth2:
      resourceserver:
        jwt:
          issuer-uri: http://localhost:8081/realms/quickbite
```

Config Spring (remplace le filtre JWT maison) + mapping des rôles Keycloak :

```
http.oauth2ResourceServer(o -> o.jwt(jwt ->
  jwt.jwtAuthenticationConverter(keycloakConverter())));

// Converter : extrait realm_access.roles -> ROLE_*
@Bean
JwtAuthenticationConverter keycloakConverter() {
  JwtAuthenticationConverter conv = new JwtAuthenticationConverter();
  conv.setJwtGrantedAuthoritiesConverter(jwt -> {
    var realm = (Map<String, Object>) jwt.getClaims().getOrDefault("realm_access", Map.of());
    var roles = (List<String>) realm.getOrDefault("roles", List.of());
    return roles.stream()
      .map(r -> new SimpleGrantedAuthority("ROLE_" + r.toUpperCase()))
      .collect(Collectors.toList());
  });
  return conv;
}
```

**Test :** obtenir un token via Keycloak, appeler `/admin/users`, décoder le token sur [jwt.io](https://jwt.io).

# Atelier 5 — Durcissement

---

**Notion (M3-J5) :** CORS, gestion centralisée des exceptions, limitation de débit, en-têtes de sécurité, audit. (Le CORS et les headers sont déjà dans `SecurityConfig` ci-dessus.)

**Fichier :** `web/GlobalExceptionHandler.java`

```
package com.quickbite.web;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.security.access.AccessDeniedException;
import org.springframework.security.authentication.BadCredentialsException;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;
import org.springframework.web.server.ResponseStatusException;
import java.util.Map;

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(BadCredentialsException.class)
    public ResponseEntity<Map<String, String>> badCredentials(BadCredentialsException ex) {
        return ResponseEntity.status(HttpStatus.UNAUTHORIZED)
            .body(Map.of("error", "Identifiants invalides"));
    }

    @ExceptionHandler(AccessDeniedException.class) // levée par @PreAuthorize -> 403
    public ResponseEntity<Map<String, String>> denied(AccessDeniedException ex) {
        return ResponseEntity.status(HttpStatus.FORBIDDEN)
            .body(Map.of("error", "Accès refusé"));
    }

    @ExceptionHandler(IllegalArgumentException.class)
    public ResponseEntity<Map<String, String>> illegalArg(IllegalArgumentException ex) {
        return ResponseEntity.status(HttpStatus.BAD_REQUEST)
            .body(Map.of("error", ex.getMessage()));
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<Map<String, String>> validation(MethodArgumentNotValidException ex) {
        return ResponseEntity.badRequest().body(Map.of("error", "Requête invalide"));
    }

    @ExceptionHandler(ResponseStatusException.class)
    public ResponseEntity<Map<String, String>> statusEx(ResponseStatusException ex) {
        return ResponseEntity.status(ex.getStatusCode())
            .body(Map.of("error", ex.getReason() == null ? "Erreur" : ex.getReason()));
    }

    @ExceptionHandler(Exception.class)
    public ResponseEntity<Map<String, String>> generic(Exception ex) {
        return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR)
            .body(Map.of("error", "Erreur interne"));
    }
}
```

**Fichier :** security/LoginRateLimitFilter.java

```

package com.quickbite.security;

import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import org.springframework.lang.NonNull;
import org.springframework.stereotype.Component;
import org.springframework.web.filter.OncePerRequestFilter;
import java.io.IOException;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.atomic.AtomicInteger;

@Component
public class LoginRateLimitFilter extends OncePerRequestFilter {

    private static final int MAX_ATTEMPTS = 5;
    private static final long WINDOW_MS = 60_000;

    private record Counter(AtomicInteger count, long windowStart) { }
    private final Map<String, Counter> buckets = new ConcurrentHashMap<>();

    @Override
    protected void doFilterInternal(@NonNull HttpServletRequest req,
                                    @NonNull HttpServletResponse res,
                                    @NonNull FilterChain chain)
        throws ServletException, IOException {
        boolean isLogin = "POST".equalsIgnoreCase(req.getMethod())
            && "/auth/login".equals(req.getServletPath());
        if (isLogin && isBlocked(clientKey(req))) {
            res.setStatus(429);
            res.setContentType("application/json");
            res.getWriter().write("{\"error\":\"Trop de tentatives. Réessayez plus tard.\"}");
            return;
        }
        chain.doFilter(req, res);
    }

    private boolean isBlocked(String key) {
        long now = System.currentTimeMillis();
        Counter c = buckets.compute(key, (k, ex) -> {
            if (ex == null || now - ex.windowStart() > WINDOW_MS)
                return new Counter(new AtomicInteger(1), now);
            ex.count().incrementAndGet();
            return ex;
        });
        return c.count().get() > MAX_ATTEMPTS;
    }

    private String clientKey(HttpServletRequest req) {
        String fwd = req.getHeader("X-Forwarded-For");
        return (fwd != null && !fwd.isBlank()) ? fwd.split(",")[0].trim() : req.getRemoteAddr();
    }
}

```



## Test du rate limiting :

```
for i in $(seq 1 6); do
  curl -s -o /dev/null -w "%{http_code}\n" -X POST http://localhost:8080/auth/login \
    -H "Content-Type: application/json" -d '{"username":"x","password":"y"}'
done
# premières tentatives -> 401, puis -> 429
```

**Check-list OWASP appliquée :** BCrypt · tokens courts + rotation · CORS restrictif · CSRF cohérent (stateless) · rate limiting · moindre privilège (RBAC) · messages neutres · secret JWT via variable d'environnement.

---

# Atelier D1 — Conteneuriser (Docker)

## Fichier : `Dockerfile`

```
# ----- build -----  
FROM maven:3.9-eclipse-temurin-21 AS build  
WORKDIR /app  
COPY pom.xml .  
RUN mvn -B dependency:go-offline  
COPY src ./src  
RUN mvn -B clean package -DskipTests  
  
# ----- runtime -----  
FROM eclipse-temurin:21-jre-alpine  
WORKDIR /app  
ENV SPRING_PROFILES_ACTIVE=postgres  
COPY --from=build /app/target/quickbite-api-*.jar app.jar  
EXPOSE 8080  
RUN addgroup -S app && adduser -S app -G app  
USER app  
ENTRYPOINT ["java", "-jar", "app.jar"]
```

## Fichier : `docker-compose.yml`

```
services:  
  api:  
    build: .  
    ports: ["8080:8080"]  
    environment:  
      SPRING_PROFILES_ACTIVE: postgres  
      DATABASE_URL: jdbc:postgresql://db:5432/quickbite  
      DATABASE_USER: app  
      DATABASE_PASSWORD: secret  
      JWT_SECRET: une-cle-secrete-de-32-octets-minimum-a-changer!!  
      CORS_ORIGINS: http://localhost:4200  
    depends_on:  
      db: { condition: service_healthy }  
    networks: [app-net]  
  db:  
    image: postgres:16-alpine  
    environment:  
      POSTGRES_USER: app  
      POSTGRES_PASSWORD: secret  
      POSTGRES_DB: quickbite  
    volumes: ["db-data:/var/lib/postgresql/data"]  
    healthcheck:  
      test: ["CMD-SHELL", "pg_isready -U app -d quickbite"]  
      interval: 5s  
      timeout: 3s  
      retries: 10  
    networks: [app-net]  
volumes: { db-data: }  
networks: { app-net: }
```

**Fichier :** `src/main/resources/application-postgres.yml`

```
spring:
  datasource:
    url: ${DATABASE_URL:jdbc:postgresql://localhost:5432/quickbite}
    username: ${DATABASE_USER:app}
    password: ${DATABASE_PASSWORD:secret}
    driver-class-name: org.postgresql.Driver
  jpa:
    hibernate:
      ddl-auto: update
    database-platform: org.hibernate.dialect.PostgreSQLDialect
  h2:
    console:
      enabled: false
```

**Test :**

```
docker compose up -d --build
curl http://localhost:8080/health
docker compose down -v
```

# Atelier D2 — Pipeline CI

---

Fichier : `.github/workflows/ci.yml`

```
name: CI
on:
  push:
    branches: [ main ]
    tags: [ 'v*' ]
  pull_request:
jobs:
  build-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with:
          distribution: temurin
          java-version: '21'
          cache: maven
      - run: mvn -B verify
  docker-publish:
    needs: build-test
    if: startsWith(github.ref, 'refs/tags/v')
    runs-on: ubuntu-latest
    permissions: { contents: read, packages: write }
    steps:
      - uses: actions/checkout@v4
      - uses: docker/login-action@v3
        with:
          registry: ghcr.io
          username: ${ github.actor }
          password: ${ secrets.GITHUB_TOKEN }
      - uses: docker/build-push-action@v6
        with:
          context: .
          push: true
          tags: ghcr.io/${ github.repository }:${ github.ref_name }
```

# Atelier D3 — Déployer sur Kubernetes

---

Fichier : `k8s/config.yaml`

```
apiVersion: v1
kind: ConfigMap
metadata: { name: quickbite-config }
data:
  SPRING_PROFILES_ACTIVE: "postgres"
  DATABASE_URL: "jdbc:postgresql://quickbite-db:5432/quickbite"
  DATABASE_USER: "app"
  CORS_ORIGINS: "https://app.quickbite.com"
---
apiVersion: v1
kind: Secret
metadata: { name: quickbite-secret }
type: Opaque
stringData:
  DATABASE_PASSWORD: "secret-a-changer"
  JWT_SECRET: "une-cle-secrete-de-32-octets-minimum-a-changer!!"
```

**Fichier :** `k8s/deployment.yaml`

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: quickbite-api
  labels: { app: quickbite-api }
spec:
  replicas: 2
  selector:
    matchLabels: { app: quickbite-api }
  template:
    metadata:
      labels: { app: quickbite-api }
    spec:
      containers:
        - name: api
          image: ghcr.io/monuser/quickbite-api:1.0.0
          ports: [{ containerPort: 8080 }]
          envFrom:
            - configMapRef: { name: quickbite-config }
            - secretRef: { name: quickbite-secret }
          readinessProbe:
            httpGet: { path: /health, port: 8080 }
            initialDelaySeconds: 10
          livenessProbe:
            httpGet: { path: /actuator/health, port: 8080 }
            initialDelaySeconds: 20
          resources:
            requests: { cpu: "200m", memory: "256Mi" }
            limits: { cpu: "750m", memory: "512Mi" }
---
apiVersion: v1
kind: Service
metadata: { name: quickbite-api }
spec:
  selector: { app: quickbite-api }
  ports: [{ port: 80, targetPort: 8080 }]
  type: ClusterIP
```

**Fichier :** `k8s/ingress.yaml`

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata: { name: quickbite-ingress }
spec:
  rules:
    - host: quickbite.local
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: quickbite-api
                port: { number: 80 }
```

**Test :**

```
minikube start
minikube addons enable ingress
kubectl apply -f k8s/config.yaml
kubectl apply -f k8s/deployment.yaml
kubectl apply -f k8s/ingress.yaml
kubectl get pods,svc,ingress
# /etc/hosts : "<minikube ip> quickbite.local", puis :
curl http://quickbite.local/health
```

---

# TP final (synthèse du cursus)

- ☒ API REST sécurisée par JWT (login/refresh, endpoints protégés)
- ☒ Console admin / RBAC (rôles & permissions, 401/403)
- ☒ (option) Intégration Keycloak
- ☒ Durcissement (CORS, exceptions, rate limit, headers)
- ☒ Conteneurisation + pipeline CI + déploiement Kubernetes
- ☒ Documentation Postman fournie : `postman/QuickBite.postman_collection.json`

## Structure du projet

```
atelier-quickbite/  
├─ pom.xml  
├─ Dockerfile .dockerignore docker-compose.yml  
├─ .github/workflows/ci.yml  
├─ k8s/ (deployment, config, ingress)  
├─ postman/QuickBite.postman_collection.json  
└─ src/main/java/com/quickbite/  
    ├─ QuickBiteApplication.java  
    ├─ config/DataInitializer.java  
    ├─ security/ (SecurityConfig, JwtService, JwtAuthFilter, JwtEntryPoint,  
    │           RestAccessDeniedHandler, LoginRateLimitFilter, SecurityProperties)  
    ├─ user/ (UserEntity, RoleEntity, repositories, JpaUserDetailsService)  
    ├─ auth/ (AuthController, AuthService, RefreshToken*, AuthDtos)  
    ├─ order/ (OrderEntity, OrderRepository, OrderController)  
    ├─ admin/ (AdminController)  
    └─ web/ (HealthController, GlobalExceptionHandler)
```